



Department of Electrical Engineering

Faculty Member: Engr. Shah Muhammad

Dated: 03rd May, 2026

Class: BEE-17-C

Semester: 2nd Semester

CS-107: Computer Programming

Project Report

Student Name	CMS ID
Zaka Ullah Khan	542210
Muhammad Safi Ullah	548923
Aakash Shahid Butt	542098
Muhammad Saim	541404



Softwares and Tools Used:

- VS Code
- C++ 17
- SFML (Simple and Fast Multimedia Library) version 3.0.0
- GCC 14.2.0 MinGW (SEH) (UCRT) 64-bit Compiler

Table of Contents

1. Introduction to SFML	3
2. Introduction to Paddle Royale	3
2.1. How the Game Works	3
2.2. Game Dimensions and Speeds	4
2.3. SFML 2D Coordinate System and Velocity Physics	4
2.4. Understanding Shape Origins (setOrigin)	5
2.5. SFML Keyboard Input: Scancodes and the QWERTY Layout	6
3. Fundamental C++ Concepts Involved	7
3.1. Variables, Data Types, and Objects	7
3.2. Enums (Enumerations)	7
3.3. Structs and Classes	7
3.4. Lambda Functions	7
4. Detailed Code Breakdown	8
5. Problem Solving: Overcoming Physics Bugs	12
6. Complete List of SFML Commands Used	14
7. Game Outputs	19
8. References	25



Two Player Brick-Breaker Game with Competitive Faceoff Mode

1. Introduction to SFML:

SFML (Simple and Fast Multimedia Library) is a free, open-source, and cross-platform API written in C++. It is designed to provide a simple bridge between developers and the computer's underlying multimedia hardware. Instead of writing complex, low-level OS-specific code, developers use SFML to effortlessly tap into hardware-accelerated 2D graphics (via OpenGL), audio playback, and window management.

In this project, SFML acts as the backbone of the entire engine. It is used to generate the fullscreen application window, render 2D geometric shapes (like the paddles, ball, bricks, and stars), handle TrueType fonts (ARIAL.ttf) to render on-screen text, process audio files for the background music, and detect real-time input from the player's keyboard and mouse.

2. Introduction to Paddle Royale:

"Paddle Royale" is meant to create a new experience in the classic retro brick-breaker genre. Traditionally, games like Breakout or Arkanoid are solitary, single-player grinds. This project flips the genre into a competitive, local multiplayer showdown.

2.1 How the Game Works

The game begins in **"Breaker Mode"**, where the screen is split in half by a center dividing line in **"Two Player Mode"**. Both players simultaneously play their own independent game of brick breaker, racing to clear their respective halves of the 12x12 brick grid. Players can hit special green bricks to increase the paddle size of their opponent, or hit red bricks that shrink their own paddle.

Once both players clear their bricks - or if one player finishes first and a 60-second waiting timer runs out - the game transitions into a **"Faceoff Mode"** with a one life penalty for the player who didn't clear their bricks under the 60 seconds timer. The center dividing line drops, and the game becomes a direct PvP battle. Players take turns launching the ball at the opponent's paddle and deplete their 6 lives.



The player with lesser number of lives goes first, otherwise a random player will go first. The last paddle standing wins.

2.2 Game Dimensions and Speeds

The game is mathematically balanced using the following physical dimensions and movement speeds:

- **UI Walls:** The dark gray side panels are 240 pixels wide and 1080 pixels tall. This squeezes the active playable arena into the exact middle 1440 pixels of a 1920x1080 screen.
- **Paddles:** The default paddle (Size Level 4) is exactly 240 pixels wide and 20 pixels tall. Hitting green/red bricks adjusts the size tier (1 through 7). A Level 1 paddle is a tiny 60 pixels wide, while a Level 7 paddle spans a massive 420 pixels. The paddle movement speed is locked at 20 pixels per frame in the Faceoff Mode and 12 pixels per frame in the Breaker Mode.
- **Ball:** The ball is a circle with a precise radius of 10 pixels (20x20 pixels total bounding box). In Breaker Mode, it naturally launches diagonally at 8 pixels per frame. During the Faceoff Mode, it launches vertically at 15 pixels per frame.
- **Bricks:** Every brick is exactly 120 pixels wide and 35 pixels tall. They are generated in a seamless 12x12 grid, giving each player an exact 6x12 wall of targets to destroy.

2.3 SFML 2D Coordinate System and Velocity Physics

Unlike traditional Cartesian coordinate system where the Y-axis goes UP, SFML (like most computer graphics libraries) places the origin point (0, 0) at the top-left corner of the window. This means the X-axis is positive to the RIGHT, and the Y-axis is positive DOWNWARD. Because our resolution is 1920x1080, the bottom-right corner of our screen rests at the coordinate (1920, 1080).



This inverted Y-axis fundamentally dictates how ball velocity (*'ballVel'*) and bouncing math work in the code:

- Moving UP means subtracting from Y (Negative Y velocity).
- Moving DOWN means adding to Y (Positive Y velocity).

For example, when the ball hits the top ceiling of the screen ($Y \leq 0$), we must reverse its direction to make it go down by forcing its Y velocity to become positive ($ballVel.y *= -1$). Conversely, when Player 1 (who sits at the bottom of the screen around $Y=1000$) hits the ball with their paddle, we must force the ball to travel UP towards the bricks. We do this by hardcoding the Y velocity to be strictly negative: $ballVel.y = -std::abs(ballVel.y)$.

2.4 Understanding Shape Origins (*setOrigin*)

By default, whenever you create a shape in SFML (like a paddle, a ball, or text), its "origin" point is located at its extreme top-left corner. This means if we want a 100-pixel wide paddle to move to coordinate $X=500$, the top-left corner of the paddle goes to 500, meaning the paddle visually stretches from 500 to 600.

In Paddle Royale, we explicitly shift the origin of the paddles and the balls to their exact geometric centers (e.g., `paddle.setOrigin({getCurrentWidth() / 2.f, 10.f})`). This shift unlocks two critical gameplay functions:

1. **Positioning Objects:** When the origin is at the default top-left corner, calling "`setPosition(960, 1000)`" places the left edge of the paddle at the center of the screen, leaving the rest of the paddle completely off to the right. By moving the origin to the center, calling "`setPosition(960, 1000)`" places the absolute middle of the paddle exactly in the middle of our 1920x1080 screen. This makes aligning UI elements and centering players mathematically effortless.
2. **Math and Trajectory Collision:** When we calculate the *'hitOffset'* to steer the ball, we simply subtract the paddle's X coordinate from the ball's X coordinate. Because we centered the origin, `paddle.getPosition().x` accurately returns the absolute physical center of the paddle.



In this way, if the ball hits the paddle on the left, it will deflect towards the left and same for the right side, essentially allowing us to steer the direction of the ball. If we had left the origin at the top-left, our math would falsely calculate the ball hitting the center of the paddle as hitting the far right side.

3. **Dynamic Resizing:** When a player hits a green or red brick, the paddles dynamically grow or shrink. If the origin was left at the top-left corner, increasing the width would make the paddle grow exclusively to the right side, drastically throwing off the player's positioning mid-game. By centering the origin, the paddle expands and contracts evenly outwards from the middle, ensuring the player's physical center remains completely stationary.

2.5 SFML Keyboard Input: Scancodes and the QWERTY Layout

When a player presses a key, the physical keyboard hardware doesn't actually send a letter (like 'A' or 'D') to the computer. Instead, it generates a raw electrical signal known as a "**Scancode**". A scancode only identifies the physical location of the key on the circuit board (e.g., "the player pressed the second key in the middle row").

The computer's Operating System intercepts this raw scancode and runs it through a translator based on the user's selected language settings. For most English-speaking users, this translation map is the standard QWERTY layout. The OS identifies at the scancode and tells SFML, "The key that was just pressed translates to the logical key 'A'."

In our engine, when we query `sf::Keyboard::isKeyPressed(sf::Keyboard::Key::A)`, we are checking for the "Logical Key", not the raw hardware scancode. Because the code is written asking for these specific logical keys, the game fundamentally assumes the player is using a standard QWERTY layout, meaning 'A' and 'D' will be perfectly positioned under Player 1's left hand. If a user had a different keyboard map (like the French AZERTY layout), the physical key located where 'A' normally is would register as 'Q', forcing them to press 'Q' and 'D' to move instead.



3. Fundamental C++ Concepts Involved:

3.1 Variables, Data Types, and Objects

The game utilizes basic variables like “int” (for discrete values like player lives), “float” (for continuous values like movement speed and coordinates), and “bool” variables for true/false flags. More importantly, it relies heavily on Object-Oriented objects provided by SFML, such as “*sf::RectangleShape*” and “*sf::CircleShape*”. These objects combine complex data (position, dimensions, color) and methods to manipulate that data on the screen.

3.2 Enums (Enumerations)

An enumeration provides a way to assign human-readable names to underlying integers. The code uses an “*enum class State*” containing values like “TITLE, BREAKER, WAITING, and FACEOFF”. This creates a highly readable “State Machine”. The main game loop can check “*if (gameState == State::TITLE)*” to instantly know which screen it should be processing, rather than relying on confusing numerical codes.

3.3 Structs and Classes

A “struct” is used to bundle related variables and functions into a single customized data type. The “*Player*” struct bundles together a paddle shape, a ball shape, velocity vectors, and remaining lives. This keeps the code incredibly clean; instead of managing isolated variables like “*player1X*” and “*player2X*”, the engine can easily manage “*p1*” and “*p2*” as distinct, self-contained structures with identical internal rules.

3.4 Lambda Functions

Lambdas are modern C++ features that allow developers to create anonymous, inline functions on the fly. The code uses lambdas like “*auto resetGame = [&]() { ... };*” to encapsulate the complex logic of resetting paddle positions and respawning bricks. Passing “[&]” means the lambda captures surrounding variables by reference, allowing it to modify the global game state cleanly without cluttering the global namespace.



4. Detailed Code Breakdown:

Snippet 1: Includes and Namespaces

```
#include <SFML/Graphics.hpp>

#include <SFML/Audio.hpp>

#include <vector>

#include <cmath>
```

Explanation: These lines import external libraries. The SFML headers pull in the graphics and audio modules. “<vector>” allows the use of dynamic arrays (which flexibly grow and shrink to hold our bricks and stars), and “<cmath>” is included for mathematical operations like absolute values and square roots required by our physics engine.

Snippet 2: User Interface - Text and Buttons

```
sf::RectangleShape btnSPRect({ 400.f, 70.f });

btnSPRect.setPosition({ 760.f, 380.f });

btnSPRect.setFillColor(sf::Color(60, 60, 60));

sf::Text btnSPText(font, "Single Player Mode", 35);

btnSPText.setPosition({ 810.f, 395.f });
```

Explanation: SFML doesn't have built-in UI buttons, so we build them manually by combining two elements. First, an “sf::RectangleShape” is created to act as the visual background of the button, given a specific width/height, X/Y screen coordinates, and a color. Second, an “sf::Text” object is created using a loaded TrueType font (ARIAL.ttf) to hold the label. We position the text slightly offset from the rectangle so it perfectly centers over the background. During the game loop, we detect mouse clicks within the bounds of that rectangle to make it act like a clickable button.



Snippet 3: Generating Bricks via Nested Loops

```
for (int i = 0; i < 12; ++i) {  
  
    for (int j = 0; j < 12; ++j) {  
  
        Brick b;  
  
        b.shape.setPosition({ i * 120.f + 240.f, j * 35.f + 100.f });  
  
        bricks.push_back(b);  
  
    }  
  
}
```

Explanation: To build a wall of bricks without writing hundreds of lines of code, we use nested for loops. The outer loop (i) represents the columns, and the inner loop (j) represents the rows. By multiplying “i” by the width of a brick (120.f) and “j” by the height (35.f), and adding a starting offset (240.f, 100.f), the code dynamically maps out a perfectly aligned grid. Every generated brick is then pushed into the “bricks” vector list to be managed by the engine.

Snippet 4: The Main Game Loop and State Machine

```
while (window.isOpen()) {  
  
    if (gameState == State::TITLE) {  
  
        // Render Title Screen  
  
    } else if (gameState == State::BREAKER) {  
  
        // Run Game Physics  
  
    }  
  
}
```



Explanation: The “*while (window.isOpen())*” block is the heart of the game. As long as the window isn't closed, this loop runs indefinitely, usually 60 times a second. Inside this loop, the State Machine dictates the flow. By checking the “*gameState*” variable, the game decides whether to process title screen inputs, run the active brick-breaker physics, or display the Game Over screen. Only one logical block executes at a time, keeping the logic cleanly separated.

Snippet 5: The Event Loop (Single-Trigger Inputs)

```
while (const std::optional event = window.pollEvent()) {  
  
    if (const auto* mouseEvent = event->getIf<sf::Event::MouseButtonPressed>())  
    {  
  
        if (btnSPRect.getGlobalBounds().contains(mousePos)) {  
  
            isSinglePlayer = true;  
  
            gameState = State::BREAKER;  
  
        }  
  
    }  
  
}
```

Explanation: Operating systems constantly generate events (clicks, key presses etc.). “*window.pollEvent()*” queries the OS queue. We use this loop for inputs that should only trigger ONCE per action (like clicking a menu button or pressing “Escape” to pause). In the snippet above, if the engine detects a left mouse click, it checks if the mouse coordinates fall inside the bounds of the “Single Player” rectangle button. If so, it instantly changes the game state, transitioning the player to the game screen.



Snippet 6: Real-Time Player Movement Inputs

```
if (sf::Keyboard::isKeyPressed(sf::Keyboard::Left)) {  
  
    p1.paddle.move({ -12.f, 0.f });  
  
}
```

Explanation: Unlike single-trigger events, paddle movement requires continuous, smooth motion. “*sf::Keyboard::isKeyPressed*” bypasses the event queue and checks the direct, real-time physical state of the keyboard hardware. If the player is holding down the Left arrow key, this if-statement evaluates to true every single frame (60 times a second), constantly subtracting 12 pixels from the paddle's X coordinate, resulting in smooth horizontal scrolling.

Snippet 7: Rendering the Graphics

```
window.clear(sf::Color(8, 8, 16));  
  
window.draw(p1.paddle);  
  
window.display();
```

Explanation: Because SFML renders frame-by-frame, the old frames must be completely erased. “*window.clear()*” wipes the hidden drawing canvas and fills it with a dark, space-blue colored background. “*window.draw()*” stamps our paddle and ball objects onto this invisible canvas. Finally, “*window.display()*” swaps the hidden canvas with the active monitor screen, allowing the human eye to perceive seamless motion.



5. Problem Solving: Overcoming Physics Bugs:

During the physics development of Paddle Royale, the game suffered from three critical mathematical flaws related to vector calculation.

Problem 1: The Horizontal Lock

Under certain bouncing conditions, the vertical (Y) speed of the ball could reach mathematically zero. Because there was no vertical push, the ball would endlessly bounce left and right between the walls, never moving up or down to reach the bricks or the paddles.

Problem 2: The Vertical Lock

Conversely, if the horizontal (X) speed dropped to zero, the ball would bounce in a perfectly straight line up and down. A player could simply sit directly under it, completely stalling out the dynamic angle requirement of a brick-breaking game.

Problem 3: Trajectory Control

Originally, the ball reflected off the paddle like a flat mirror. However, modern arcade games require the player to "steer" the ball depending on where it strikes the paddle (e.g., hitting the paddle on the far left edge should sharply throw the ball horizontally to the left).

The Solution: Paddle Normalization and Vector Limits

To solve Problem 3 (Trajectory Control), the “*handlePaddleCollision()*” method was written with this specific algorithm:

```
float hitOffset = ball.getPosition().x - paddle.getPosition().x;
```

```
float normalizedOffset = hitOffset / (getCurrentWidth() / 2.f);
```

```
ballVel.x += normalizedOffset * 4.0f;
```



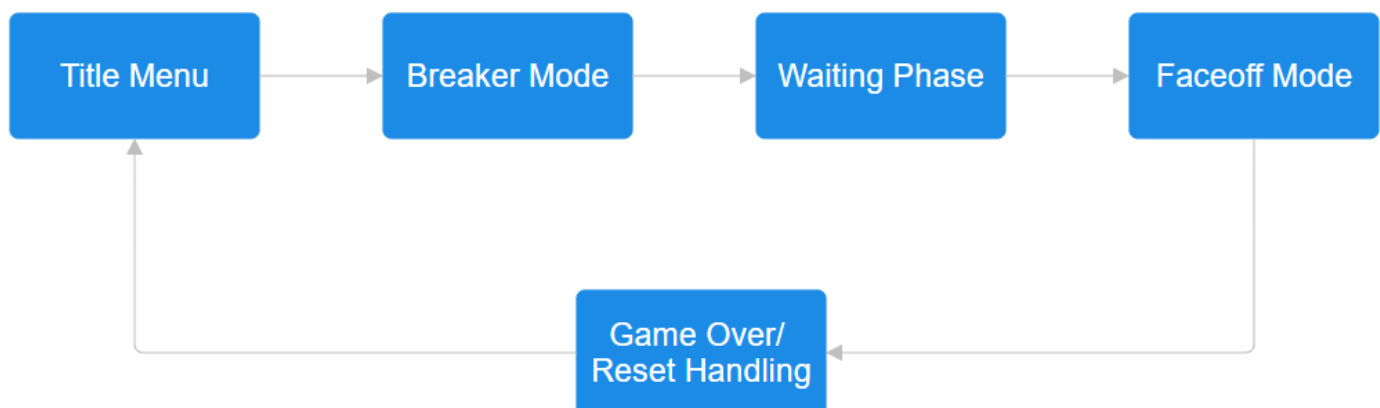
Explanation: Line 1 subtracts the coordinates to calculate exactly how many pixels away from the center of the paddle the ball struck. Line 2 divides this distance by half the paddle's width, normalizing the result into a clean ratio between -1.0 (far left) and 1.0 (far right). Line 3 applies this ratio as a multiplier to modify the ball's X velocity. This allows players to actively influence and "steer" the ball's horizontal speed.

To solve Problems 1 and 2 (The Axis Locks), the following function was implemented:

```
void enforceMinimumVerticalAngle(sf::Vector2f& vel, float speed) { ... }
```

Explanation: Every time the ball bounces off the paddle, Pythagorean theorem ($A^2 + B^2 = C^2$) is used to calculate the total speed magnitude. The vector is then pushed into “*enforceMinimumVerticalAngle()*”. This function explicitly checks if the new Y velocity is dangerously close to zero. If it is, the code artificially boosts the Y velocity up to a minimum safe threshold. It then recalculates the X velocity backward to ensure the overall total speed of the ball remains identical. This completely eradicates the possibility of the ball ever becoming stuck in an endless horizontal or vertical flatline!

NOTE: The complete game code has been documented with proper indentation and comments. For further understanding of code, kindly refer to Paddle_Royale.cpp





6. Complete List of SFML Commands Used:

The following is a comprehensive list of the exact SFML classes, objects, and functions we utilized to bring the game to life, categorized by their functionality:

a) Window and Core System Operations

- `sf::RenderWindow window(sf::VideoMode({1920, 1080}), "Title", sf::State::Fullscreen);`

Creates the main application window with specific dimensions and state (Fullscreen).

- `window.setFramerateLimit(60);`

Locks the game's execution speed to 60 frames per second to stabilize physics.

- `window.isOpen()`

Returns a boolean (true/false) checking if the window is currently active.

- `window.clear(sf::Color(8, 8, 16));`

Wipes the previous frame completely clean, replacing it with a custom background color.

- `window.draw(object);`

Draws a specific shape, sprite, or text block to the hidden rendering canvas.

- `window.display();`

Swaps the hidden rendering canvas to the active monitor, making the frame visible.

- `window.close();`

Safely terminates the window and application process.



b) Graphics, Shapes, and Colors

- `sf::RectangleShape shape({width, height});`

Creates a 2D rectangle object with specified dimensions (used for paddles, bricks, and buttons).

- `sf::CircleShape circle(radius);`

Creates a 2D circle object with a specified radius (used for the ball and stars).

- `shape.setPosition({x, y});`

Teleports a shape to exact X and Y coordinates on the screen.

- `shape.move({offsetX, offsetY});`

Slides a shape from its current position by a specific relative amount.

- `shape.setOrigin({x, y});`

Changes the internal anchor point of a shape. We set this to the center of objects for accurate math and resizing.

- `shape.setFillColor(sf::Color::White);`

Paints the interior of a shape. We also used custom RGB/Alpha combinations like `sf::Color(30, 30, 30, 230)`.

- `shape.setOutlineThickness(-3.f);`

Adds a border to a shape. A negative value pushes the border inward, which we used to give the bricks their segmented look.



c) Math, Vectors, and Collision Detection

- *sf::Vector2f ballVel{x, y};*

A data structure specifically designed to hold two floating-point numbers. We used this heavily for managing 2D velocities.

- *shape.getGlobalBounds();*

Calculates the outer bounding box of a shape. This returns an *sf::FloatRect* object which is critical for physics.

- *bounds1.findIntersection(bounds2);*

Checks if two shapes are physically overlapping. We used this to detect when the ball hits paddles and bricks.

- *bounds.contains(mousePos);*

Checks if a single coordinate point falls inside a rectangle. We used this to detect mouse hover effects and button clicks.

d) Text and Fonts

- *sf::Font font; \n font.openFromFile("C:\\\\Windows\\\\Fonts\\\\arial.ttf");*

Creates a *font* object and loads a TrueType Font file from the computer's hard drive.

- *sf::Text myText(font, "Hello", 50);*

Creates a visible text object linked to the loaded font, providing a default string and font size.

- *myText.setString("New Words");*

Dynamically updates the string currently displayed on the screen (used for life counters and timers).



- `myText.getLocalBounds();`

Calculates the exact width and height of the rendered text block. We used this to perfectly center text inside of our UI buttons.

e) Inputs (Keyboard, Mouse, and Events)

- `window.pollEvent()`

Queries the operating system for the latest user interaction event (like clicking the exit button or a single key press).

- `event->getIf<sf::Event::MouseButtonPressed>()`

Checks if the current event pulled from the queue was a mouse click.

- `event->getIf<sf::Event::KeyPressed>()`

Checks if the current event pulled from the queue was a single key press (like pressing Space to launch the ball).

- `sf::Keyboard::isKeyPressed(sf::Keyboard::Key::Left)`

Checks the immediate physical state of the keyboard to see if a key is currently held down (used for smooth, continuous paddle movement).

- `window.mapPixelToCoords(sf::Mouse::getPosition(window));`

Retrieves the raw pixel coordinates of the mouse cursor and securely maps them to the game window's coordinate system.



f) Time and Audio

- `sf::Clock waitClock; \n waitClock.restart();`

Creates an internal stopwatch object and resets it to zero. We used this to manage the 60-second waiting phase transition.s

- `waitClock.getElapsedTime().asSeconds();`

Returns the current amount of elapsed time (in seconds) as a floating-point number.

- `sf::Music bgMusic; \n bgMusic.openFromFile("Snowy.mp3");`

Creates a music streamer object and loads a large audio file from the hard drive.

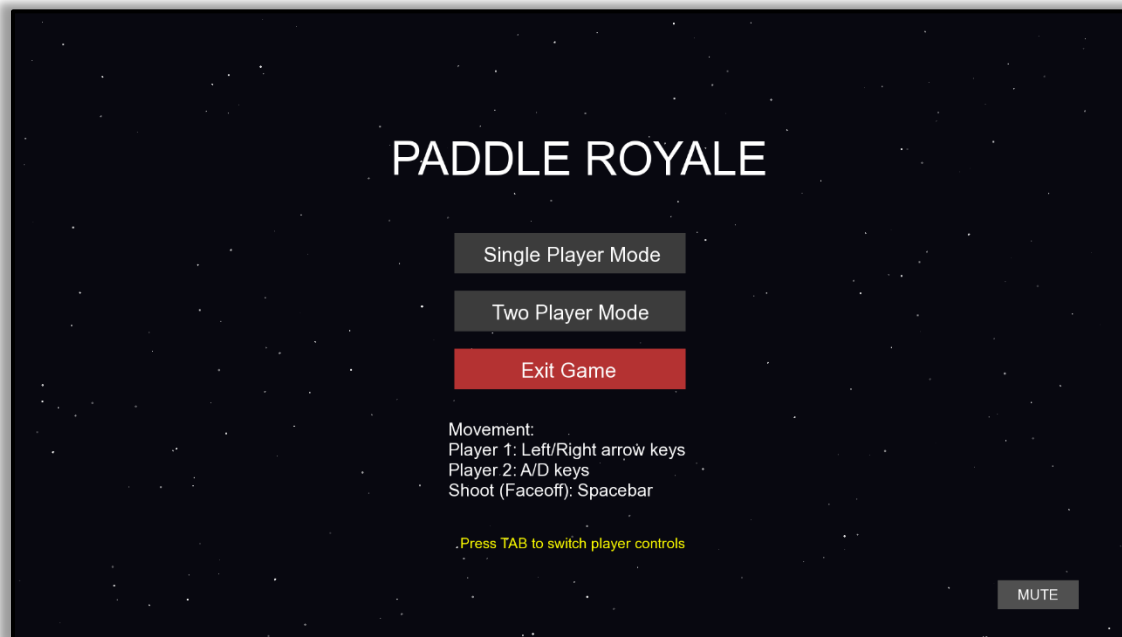
- `bgMusic.setLooping(true); \n bgMusic.setVolume(50.f); \n bgMusic.play();`

Configures the audio to repeat indefinitely, lowers the volume to 50%, and begins active playback.

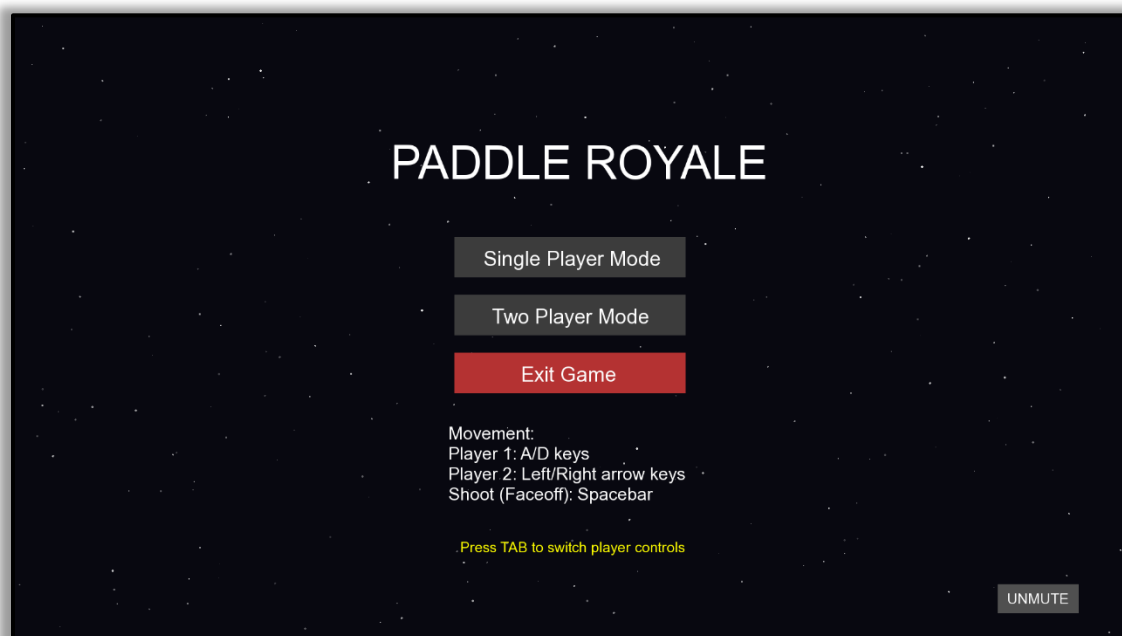


7. Game Outputs:

a) Title Screen

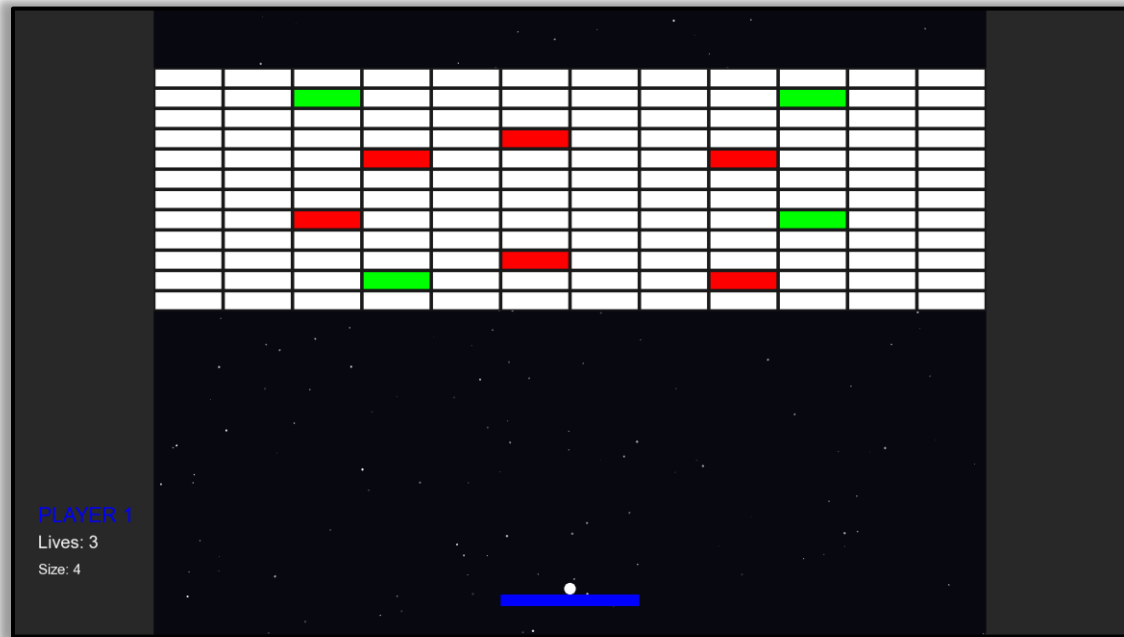


b) Controls Swapped and Music Muted

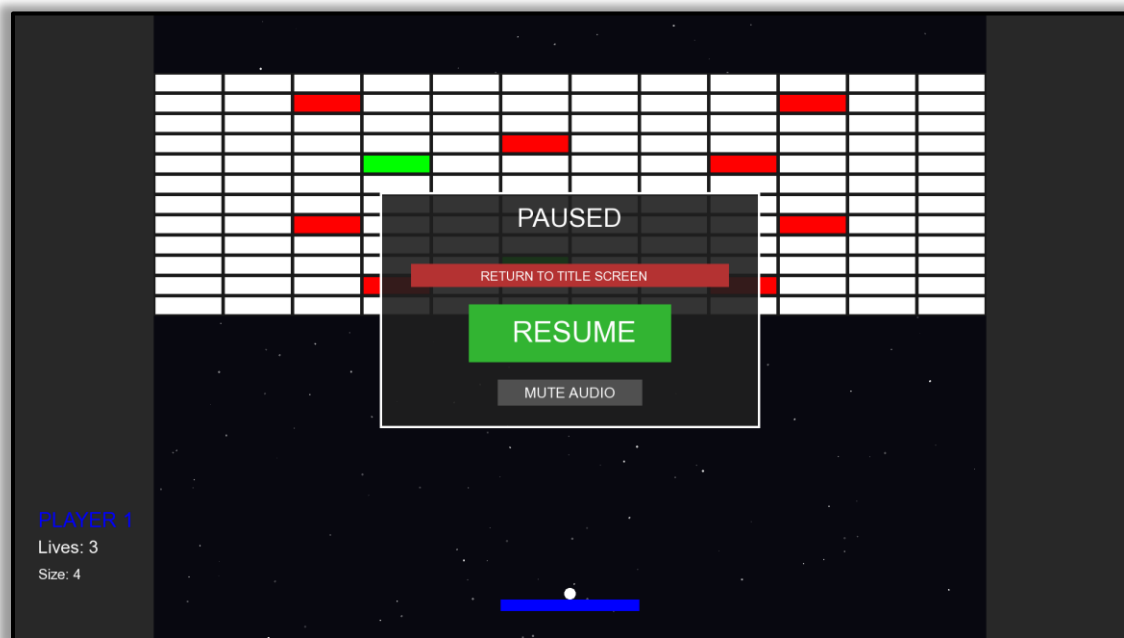




c) Single Player Mode

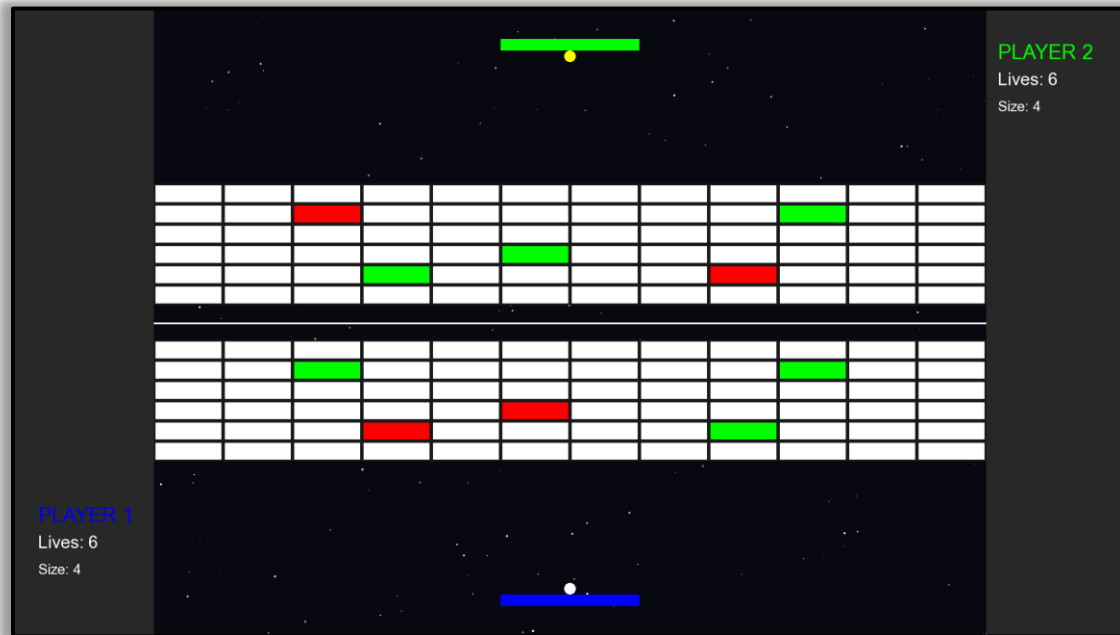


d) Single Player Pause Menu

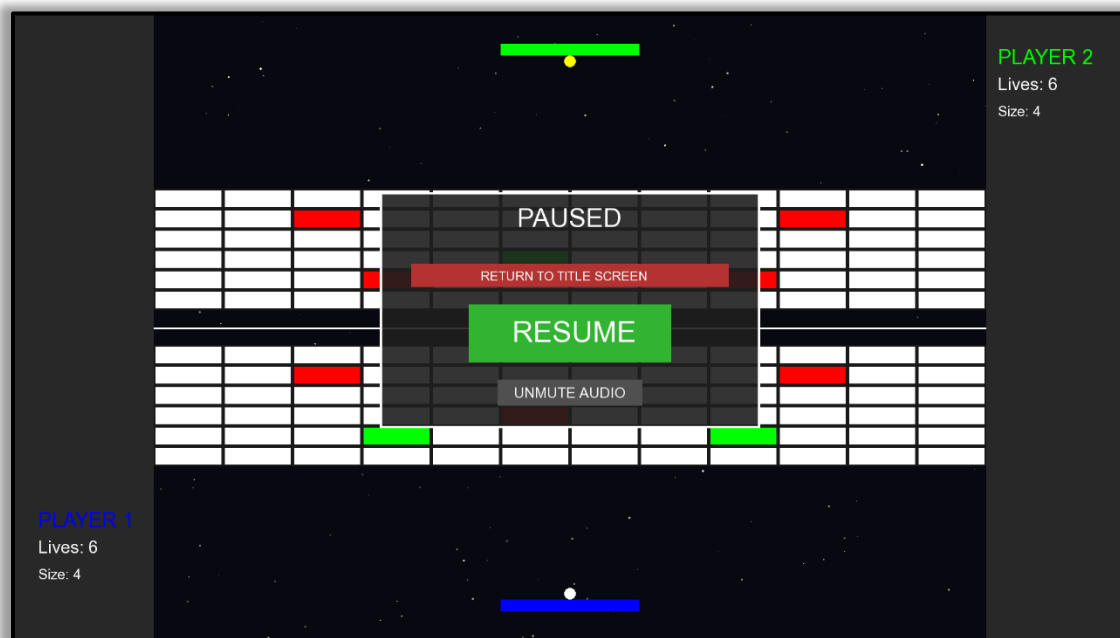




e) Two Player Mode



f) Two Player Pause Menu (Music Muted)





g) Faceoff Mode (Player 1's turn)

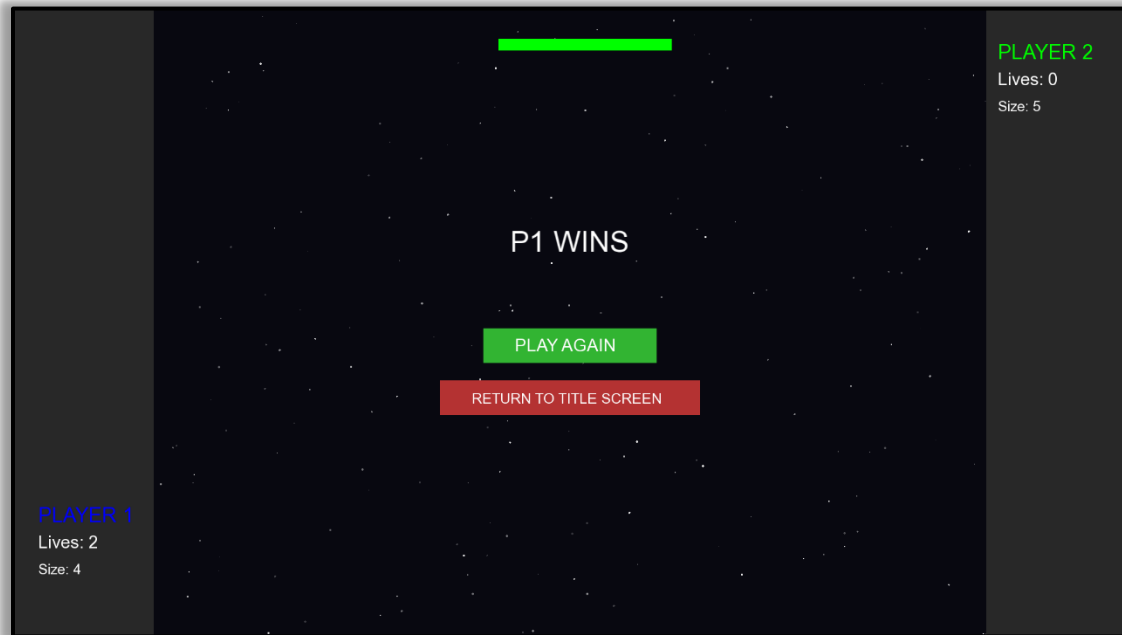


h) Faceoff Mode (Player 2's turn)





i) Game Over Screen (Player 1 wins)

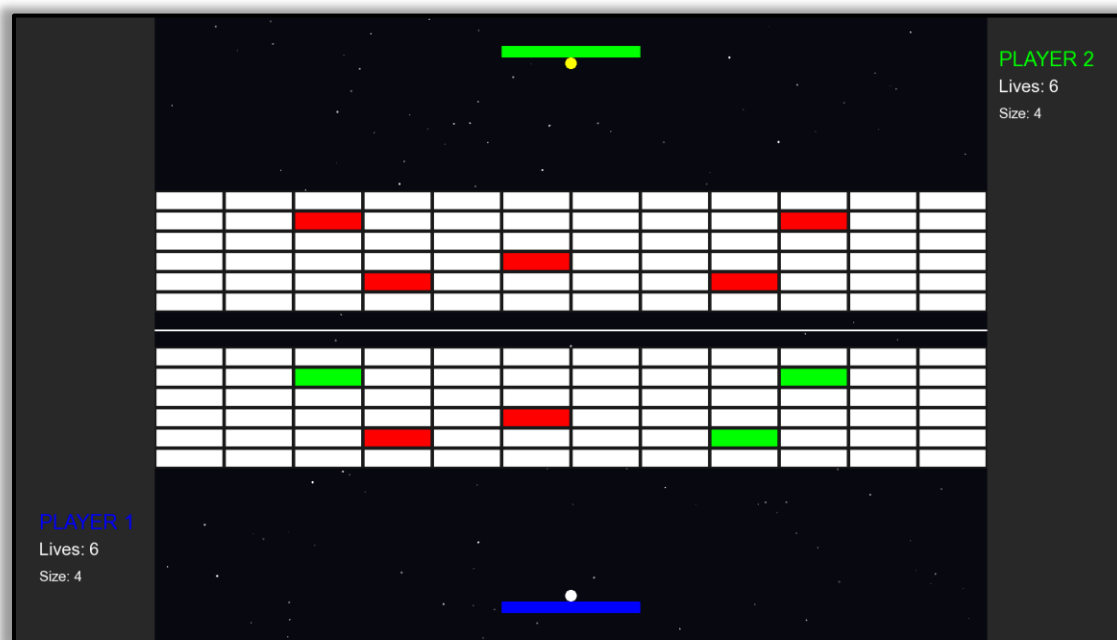


j) Waiting for other player to finish their bricks (60 seconds count-down)

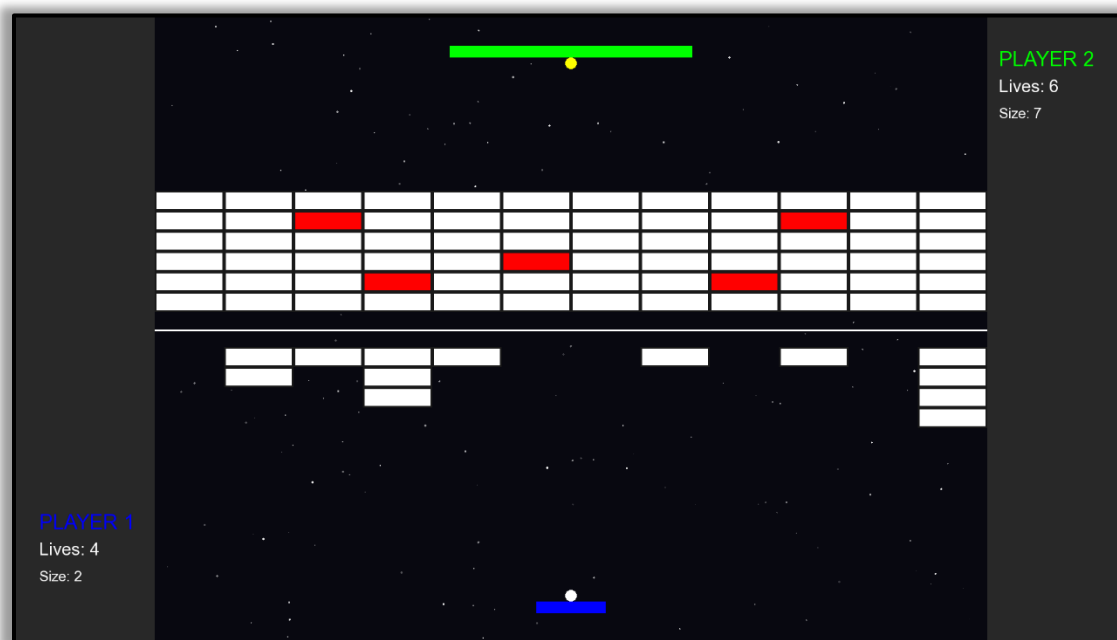




k) Default Paddle Size



l) Effect of Green and Red Bricks on Paddle Size





8. References:

<https://www.sfml-dev.org/download/sfml/3.0.0/>

<https://www.sfml-dev.org/tutorials/3.0/>

<https://youtube.com/@hopzbie?si=IxnvLjSZ8Vuz0FlM>

<https://github.com/hopzbie>

<https://youtube.com/@surajsharmafourknob?si=nwrLaRltNJHgoOI8>